

Project 12 (Bonus) — MCPResearch-Multi-Agent-Pipeline-Exposed-As-One-MCP-Tool

Title: MCPResearch Multi Agent Pipeline Exposed As One MCP Tool

Type: MCP Server (1 Tool, wrapping a 2-agent pipeline) · **Stack:** Python, official `mcp` SDK (FastMCP), LangChain (`langchain-openai`), `asyncio` · **Level:** Advanced (bonus / portfolio project) **Tagline:** A Researcher agent and a Fact-Checker agent loop until a summary is approved — the whole loop hides behind one MCP tool, `deep_research(topic)` , so any MCP client calls it like a single plain function.

Not part of the numbered Doc01-19 project arc — a bonus/portfolio project that combines two things you've already built: MCP server-building from [project_7_mcpforge](#) and Doc06's MCP Core Concepts, and the generator → critic multi-agent loop from [project_4_contentforge](#) and [11_multi_agent_systems](#). Read both first if you haven't — this file has the full build spec, not a pointer elsewhere.

Charter (what this project is)

Every other MCP project in this curriculum exposes one plain function as a Tool (Project 7), or has an agent consume someone else's MCP tools (Project 8, Project 9). This project does the opposite. It wraps a whole multi-agent pipeline — a Researcher agent that drafts a summary, a Fact-Checker agent that checks the draft's claims, and a bounded revision loop between them — and exposes the entire thing as **one single MCP tool**: `deep_research(topic) -> str` . A client that calls this one tool has no idea a 2-agent loop with revision rounds ran underneath it. It just gets back a fact-checked summary. This is "agent-as-a-service" — a real pattern used in production AI systems today, where a whole pipeline is called the exact same simple way as any other MCP tool.

Jump to: [Setup](#) · [Step 1](#) · [Step 2](#) · [Step 3](#) · [Step 4](#)

The Story — what you're actually building

Imagine you ask a sharp coworker, "what's the real story with X?" They walk off, and a few minutes later they come back with three clean sentences you can trust. You never see them write a shaky first draft. You never see them catch their own mistake, go check it again, and quietly fix it before showing you anything. You just see the finished, checked answer. That's what `deep_research(topic)` does for any MCP client that calls it.

Underneath, two small agents do the real work. The **Researcher** writes a short summary of the topic. The **Fact-Checker** reads that summary and looks for real problems — a wrong date, an invented fact, a claim stated more confidently than the evidence supports. If the Fact-Checker finds a problem, it sends the draft back with a short list of exactly what to fix. The Researcher tries again, using that feedback. This repeats until the Fact-Checker approves the draft, or until a hard cap on rounds is reached — the same generator → critic loop Project 4 already taught you, just with two roles you build fresh here (Researcher, Fact-Checker) instead of Writer and Reviewer.

The `pipeline.py` module runs that loop and returns one clean result. Everything else in this project is about the layer on top of it: `mcp_server.py` wraps that one function, `run_pipeline()`, as a single MCP tool. From outside, `deep_research("Retrieval-Augmented Generation")` looks exactly as simple as Project 7's `add_note("Buy milk")` — a name, one typed argument, a string back. Nothing in that interface hints that two agents just argued about a fact for two rounds to produce it.

The four steps build this in the order any real engineer would: Step 1 proves the two-agent pipeline works as a plain Python program, with no MCP anywhere yet — so if something's wrong later, you already know it isn't the agents. Step 2 wraps that proven pipeline in the thinnest possible MCP layer. Step 3 deals with a real problem this creates: a caller waiting on a slow, multi-round tool call with zero feedback until it's done. Step 4 deals with the other real problem: what to return when the pipeline is too slow, or breaks outright, instead of leaving the caller hanging or getting nothing back at all.

Before you read further — think about it yourself: you could expose `run_researcher` and `run_fact_checker` as two separate MCP tools instead of hiding them behind one `deep_research` tool, and let the *client* drive the back-and-forth loop itself, one tool call at a time. What would a client gain from that? What would it lose, compared to calling one tool and getting a finished, checked answer? And if `deep_research` is going to take 30 seconds because of 3 revision rounds, what should a client actually see while it waits — nothing at all, or something? Sit with both questions before you read the Steps below.

What you're actually building, in one line: a Researcher-and-Fact-Checker loop wrapped so completely behind one MCP tool that calling it looks exactly like calling any single plain function.

Why this needs to exist: real pipelines are often too complex for every caller to reimplement the looping and checking logic themselves — hiding that behind one callable tool means any client gets the same finished, checked answer without needing to know how it was made.

When you'd reach for this at a real job: any time you want to offer a whole internal workflow — a generate-then-verify process, a multi-step approval chain — as one capability other teams' tools can call, without those tools needing to understand or depend on its inner steps.

How it works, mechanically: `pipeline.py` runs the Researcher/Fact-Checker loop and returns one result; `mcp_server.py` wraps that single function call as one MCP tool, so the loop itself never shows up in the interface at all.

Why not just do it some simpler/different way: as the callout above already asks, you could expose the Researcher and the Fact-Checker as two separate MCP tools and let the client drive the loop itself. But then every client has to reimplement the looping, the round cap, and the decision of when to stop — and a badly-written client could loop forever or stop too early. A client calling one tool doesn't need to know or care that a loop ran underneath it at all — that is the entire point of "agent-as-a-service."

Where This Fits

This is a portfolio/bonus project, and it's the reverse-direction companion to two other bonus projects. Project 7 built an MCP server exposing small, single-purpose Tools — `add_note`, `search_notes` — where each tool call does one small, fast thing. Project 9 builds agents that *consume* MCP tools other people built. This project goes a third way, opposite to both: instead of exposing small functions, or consuming someone else's tools, it takes an entire multi-agent **pipeline** — the kind of thing Project 4 built as a whole standalone system — and hides it behind the interface of a single MCP tool.

From the outside, `deep_research(topic)` looks exactly like `add_note(text)` or `search_notes(query)`: a name, typed arguments, a string comes back. Nothing about the interface tells a client that a 2-agent loop with revision rounds is running underneath it. That is the whole lesson of this project — MCP doesn't care how much work happens inside a tool call. One line of Python, or an entire agent pipeline, look identical from the client's side. Being able to point at that on purpose, and defend the design choices it forces (Steps 3 and 4 below), is a specific, checkable, senior-sounding line for a portfolio: "I wrapped a multi-agent pipeline as a single callable MCP service, with progress reporting and a timeout fallback."

Setup (do this once, before Step 1)

These are the very first commands to run. You don't need to read anything else first.

```
cd project_12_mcpresearch
python -m venv .env
source .env/bin/activate          # Windows: .env\Scripts\activate
pip install "mcp[cli]" langchain langchain-openai python-dotenv pydantic
pip freeze > requirements.txt
```

Create a file called `.env` in this folder (never commit this file to git):

```
LOG_LEVEL=INFO
OPENAI_API_KEY=sk-your-real-key-here
MAX_REVISION_ROUNDS=3
DEEP_RESEARCH_TIMEOUT_SECONDS=60
```

Create a second file, `.env.example`, with the same keys but a placeholder for the real secret:

```
LOG_LEVEL=INFO
OPENAI_API_KEY=sk-your-key-here
MAX_REVISION_ROUNDS=3
DEEP_RESEARCH_TIMEOUT_SECONDS=60
```

`OPENAI_API_KEY` is a real secret — only `.env` should have your real value, and only `.env.example` goes in git.

Add `.venv/` and `.env` to your `.gitignore` before your first commit.

`mcp[cli]` installs the **MCP Inspector**, the same tool you used in Project 7. Once `mcp_server.py` exists (from Step 2 on), you can test `deep_research` by hand with `mcp dev mcp_server.py`, without writing a client script.

A Real Example (so this isn't just theory)

Use this topic, or make your own, but keep the shape the same: something with a real, checkable fact buried in it, so the Fact-Checker's rejection isn't hypothetical.

Topic: "Retrieval-Augmented Generation (RAG)"

Round 1: The Researcher drafts something like: *"Retrieval-Augmented Generation (RAG) was introduced by OpenAI in 2019 to let language models pull in outside documents before answering."* The Fact-Checker rejects it: `REJECTED` — issue: *"RAG was introduced by Lewis et al. at Facebook AI Research (now Meta AI) in 2020, not OpenAI in 2019."*

Round 2: The Researcher redrafts using that exact feedback: *"Retrieval-Augmented Generation (RAG) was introduced by Lewis et al. at Meta AI (then Facebook AI Research) in 2020. It lets a language model retrieve relevant documents before generating an answer, instead of relying only on what it memorized during training."* The Fact-Checker approves it.

What `deep_research("Retrieval-Augmented Generation (RAG)")` returns (from Step 3 onward) — the final summary, plus a short trace of what happened:

```
Deep research on: Retrieval-Augmented Generation (RAG)

Revision trace:
```

Round 1: rejected

- RAG was introduced by Lewis et al. at Facebook AI Research (now Meta AI) in

Round 2: approved

Status: approved after 2 round(s).

Summary:

Retrieval-Augmented Generation (RAG) was introduced by Lewis et al. at Meta AI (then Facebook AI Research) in 2020. It lets a language model retrieve relevant documents before generating an answer, instead of relying only on what it memorized during training.

Two more topics worth testing on purpose: - A topic with almost nothing checkable in it (e.g. "my opinion of the color blue") — watch what happens when the Fact-Checker has nothing real to verify. Does it approve immediately, or nitpick forever? This is the case that proves your revision cap actually matters. - A topic you deliberately make the pipeline slow on, or a fake network failure inside `run_researcher` — this is what Step 4's timeout and fallback path are for. If you never test the unhappy path, you don't actually know it works.

Why This Project Is Good for Your Portfolio

- **What problem it solves:** shows a real client — any MCP client, not a custom integration — getting a fact-checked answer from a multi-agent system through one plain function call, with no idea of the machinery underneath.
- **Why it matters:** "agent-as-a-service," callable through a standard protocol, is a real and increasingly common production pattern. It's a different, harder skill than either building a simple MCP server (Project 7) or building an agent that calls one (Project 9) — you're the one deciding what a slow, multi-step process should look like from the outside.
- **When you'd build something like this at a real job:** any time you want to offer a whole internal workflow — a research pipeline, a multi-step approval chain, a generate-then-verify process — as one callable capability other teams' tools or agents can use, without exposing (or letting them depend on) its internal steps.
- **How it's built:** a `FastMCP` server exposing exactly one Tool, `deep_research`, whose body calls a Researcher/Fact-Checker loop capped at a fixed number of rounds, reports its progress as a trace inside the result, and falls back to a clearly-marked partial answer if it times out or a step fails.

Problems you'll likely run into, and how to fix them:

Problem	Fix
The Fact-Checker approves everything on the first try, so the loop never actually runs more than once	Your Fact-Checker's prompt is too soft. Write real scoring criteria into it (dates, names, invented facts, overconfident claims), the same lesson Project 4's Reviewer prompt taught — a vague critic approves almost everything
The pipeline loops the full <code>MAX_REVISION_ROUNDS</code> every single time, never converging	The Researcher isn't actually using the feedback. Check that <code>feedback</code> is really being passed into the prompt each round, and that the prompt tells it to fix only the named issues, not rewrite from scratch
<code>deep_research</code> hangs the MCP Inspector for a long time with no feedback	Expected before Step 3 — that's exactly the problem Step 3 fixes, by returning a revision trace alongside the answer instead of only the bare final string
A slow topic makes <code>deep_research</code> hang seemingly forever	Expected before Step 4 — Step 4 adds a real timeout and a "best draft so far" fallback, so the tool always returns <i>something</i>
The client hangs and never gets a reply, even though your code looks right	Something is writing to stdout directly (a stray <code>print()</code>) — same stdio-transport rule as Project 7: stdout is the protocol channel. Route all logging through <code>logging_setup.py</code> 's logger, never <code>print()</code>
The MCP tool still "hangs" even after adding <code>asyncio.wait_for()</code> in Step 4	You wrapped a blocking, synchronous call directly with <code>wait_for</code> — a plain sync function inside an <code>async def</code> still blocks the event loop. Run it with <code>asyncio.to_thread()</code> first, then wrap that in <code>wait_for()</code>

Built During These Documents

[06_tools_function_calling](#) (MCP Core Concepts) → [11_multi_agent_systems](#) (the generator → critic pattern), building directly on [project_7_mcpforge](#) and [project_4_contentforge](#).

Plan Before You Code

Same process as every project (see [15_five_projects_index](#)): Problem → Requirements → Architecture → Components → Data Flow → Implementation Plan → Coding Tasks. Write it down yourself before you open your editor. For this project specifically, decide up front which layer owns which job — the Researcher's job, the Fact-Checker's job, the pipeline's job (looping and capping), and the MCP layer's job (formatting, timeouts, fallbacks) — before you write a single function. Mixing those up (for example, putting a timeout inside the Researcher instead of around the whole pipeline call) is the most common design mistake in a project shaped like this one.

How To Build This — Step by Step

Step 1 — The Two-Agent Pipeline, Running Standalone (No MCP Yet)

Project: MCPResearch-Multi-Agent-Pipeline-Exposed-As-One-MCP-Tool — Step 1 of 4: The Two-Agent Pipeline, Running Standalone (No MCP Yet)

What this step does: builds the Researcher, the Fact-Checker, and the loop between them as a plain Python program you run directly — no MCP anywhere in this step. It proves the pipeline itself works before anything wraps it. **Why this step matters:** if the pipeline's own logic is broken, wrapping it in MCP later would only hide that bug behind a protocol, not fix it. Proving it alone first means any bug you hit in Step 2 onward can never be blamed on the Researcher or the Fact-Checker. **When you'll hit this for real:** any time you're about to expose something over a protocol (MCP, a REST API, a queue) — prove the thing underneath actually works on its own first, the same instinct Project 7 used for its SQLite store before adding Tools on top of it. **Read first:**

[11_multi_agent_systems Core Concepts](#) — "Every pattern below: what / why / when / trade-off" (the Generator → Critic → Revision pattern, and its "without a hard limit" warning) and "Cost and latency: multi-agent is not free."

Stuck on this step? [Hint 1](#) · [Hint 2](#) · [Show me the solution](#)

What to do: 1. Build `config.py` and `logging_setup.py` (or reuse them from an earlier project if you already have working copies). `LOG_LEVEL`, `OPENAI_API_KEY`, and `MAX_REVISION_ROUNDS` are all you need this step. 2. Write `researcher.py`: a `Draft` type (topic + summary), and `run_researcher(topic: str, feedback: str | None = None) -> Draft`. When `feedback` is given, the prompt must tell the model to fix exactly the named issues, not rewrite the whole thing from scratch — this is what makes Round 2 actually different from Round 1, the same lesson Project 4's Writer taught with its own `feedback` parameter. 3. Write `fact_checker.py`: a `FactCheckVerdict` type (`approved: bool`, `issues: list[str]`), and `run_fact_checker(draft: Draft) -> FactCheckVerdict`. Write real scoring criteria into its prompt on purpose — wrong dates, invented facts, overconfident claims — a vague prompt approves almost everything, and then you'll never actually see the loop run more than once. 4. Write `pipeline.py`: `run_pipeline(topic: str, max_rounds: int) -> PipelineResult`, where `PipelineResult` holds the final summary, whether it was approved, and a list of what happened each round. Loop: Researcher drafts → Fact-Checker checks → if approved, stop; if not, feed the issues back to the Researcher as `feedback` and go again — up to `max_rounds` times, then stop regardless, exactly like Doc11's warning about a loop that never converges. 5. Write `main.py` that calls `run_pipeline()` directly on 2-3 topics (including one deliberately hard to fact-check) and prints every round's result. Confirm the loop actually improves a deliberately wrong first draft, and confirm it stops cleanly at the cap on the hard topic instead of looping forever.

Your files after Step 1:

```

project_12_mcpresearch_agentic_mcp_tool/
├─ .env / .env.example      → LOG_LEVEL, OPENAI_API_KEY, MAX_REVISION_ROUNDS, DEEP
├─ config.py                → load_config() -> Config
├─ logging_setup.py         → get_logger(name)
├─ researcher.py            → Draft, run_researcher(topic, feedback=None) ->
├─ fact_checker.py          → FactCheckVerdict, run_fact_checker(draft) ->
├─ pipeline.py              → RevisionRound, PipelineResult, run_pipeline
└─ main.py                  → runs the pipeline standalone on test top

```

Step 2 — Wrap It as a Single MCP Tool

Project: MCPResearch-Multi-Agent-Pipeline-Exposed-As-One-MCP-Tool — Step 2 of 4: Wrap It as a Single MCP Tool

What this step does: builds `mcp_server.py`, exposing `deep_research(topic: str) -> str` as one `FastMCP` tool. The tool's whole body is a call to Step 1's `run_pipeline()` — nothing about the pipeline changes. **Why this step matters:** this is the actual lesson of the project — the MCP layer is thin. It does not know, and does not need to know, that multiple agents ran underneath. Writing this step is what proves that claim in real code instead of just an idea. **When you'll hit this for real:** any time you take something that already works (a script, a pipeline, a whole system) and expose it as one callable unit for other tools to use, without those tools needing to understand its internals. **Read first:** [06_tools_function_calling Core Concepts](#) — "What MCP actually is, and the problem it solves," "MCP servers and clients, and how they actually connect," and "Why MCP matters for the multi-agent systems this curriculum builds" — that last one describes exactly what you're doing in this step.

Stuck on this step? [Hint 1](#) · [Hint 2](#) · [Show me the solution](#)

What to do: 1. Write `mcp_server.py`: a `FastMCP("deep-research-service")` instance, and one tool, `deep_research(topic: str) -> str`, whose body is exactly two lines — call `run_pipeline(topic, max_rounds=config.max_revision_rounds)`, then return `result.final_summary`. Resist the urge to add anything cleverer yet — that's Step 3's job. 2. Write a clear tool description (a real docstring) — Doc06's "a tool's description is really a prompt" idea applies here just as much as it did to a small tool in Project 7, maybe more, since this tool's name doesn't make its cost (multiple LLM calls, possibly slow) obvious to whoever's calling it. 3. Test it with the MCP Inspector: `mcp dev mcp_server.py`, then call `deep_research` with your Real Example topic. Confirm you get back the plain final summary — no trace yet, that's Step 3. 4. Confirm `mcp.run(transport="stdio")` is the last line that actually executes, and that nothing in `researcher.py`, `fact_checker.py`, or `pipeline.py` ever calls `print()` — same stdout rule as Project 7, now with more files that could accidentally break it.

Your files after Step 2:

```

project_12_mcpresearch_agentic_mcp_tool/
├── .env / .env.example
├── config.py
├── logging_setup.py
├── researcher.py                (unchanged from Step 1)
├── fact_checker.py             (unchanged from Step 1)
├── pipeline.py                 (unchanged from Step 1)
├── main.py                     (unchanged from Step 1)
└── mcp_server.py               → FastMCP("deep-research-service"); on

```

Step 3 — Report Internal Progress Without Breaking the One-Tool Illusion

Project: MCPResearch-Multi-Agent-Pipeline-Exposed-As-One-MCP-Tool — Step 3 of 4: Report Internal Progress Without Breaking the One-Tool Illusion

What this step does: changes `deep_research` 's return value so it includes a short trace of what happened each round, alongside the final summary — not just the bare final string Step 2 returned.

Why this step matters: a caller waiting on `deep_research()` through 2-3 revision rounds currently gets no feedback at all until it's fully done. For a slow tool, that's a bad experience — the caller can't tell "still working" from "stuck." This step decides, on purpose, how to fix that without turning one simple tool call into a more complicated multi-step protocol. **When you'll hit this for real:** any tool wrapping a slow, multi-step process — a long-running job, a multi-agent pipeline, a batch operation — needs a real answer to "what does the caller see while this runs, and what does it see once it's done?" **Read first:** [06_tools_function_calling Core Concepts](#) — "MCP servers and clients, and how they actually connect" (the transport `deep_research` 's result would have to travel over) — and [11_multi_agent_systems Core Concepts](#) — "Shared state design: what goes in the shared state, and what doesn't" (the same "what's worth surfacing vs. what's just noise" decision, one level up).

Stuck on this step? [Hint 1](#) · [Hint 2](#) · [Show me the solution](#)

The decision to make, and the reasoning behind it: the official `mcp` SDK does have a live progress-reporting mechanism, but it only works end to end if the specific client calling your tool asked for progress updates and is actively listening for them — most simple clients (including the one you could write following Project 7's Step 4) don't wire that up, and even when a client does, nothing about a progress *notification* sticks around afterward as part of the actual result. This project uses a **structured result** instead: `deep_research` still returns one plain string (the interface doesn't change at all), but that string now contains a short revision-history trace above the final answer. It works with every MCP client, with no special listening code required, and — unlike a stream of progress notifications — it's still there if you look at the result five minutes later.

What to do: 1. In `mcp_server.py`, write `format_result(result: PipelineResult) -> str`: a plain text layout with the topic, one line per round (`Round 1: rejected` plus its issues, `Round 2: approved`), a status line (approved after N rounds, or "NOT approved — hit the N-round cap"), and

the final summary at the bottom. Match the shape in this README's "A Real Example" section. 2. Update `deep_research` to call `format_result(result)` instead of returning `result.final_summary` directly. 3. Test with the Inspector: call `deep_research` on your Real Example topic and confirm the trace shows both rounds, including the rejection issue from Round 1. 4. Test on the "almost nothing checkable" topic from the Real Example section too — confirm the trace clearly shows whether it converged quickly or hit the cap, so a caller reading the result can tell the difference between "this was fine" and "this struggled."

Your files after Step 3:

```
project_12_mcpresearch_agentic_mcp_tool/
├── .env / .env.example
├── config.py
├── logging_setup.py
├── researcher.py
├── fact_checker.py
├── pipeline.py                      (unchanged from Step 1)
├── main.py                        (unchanged from Step 1)
└── mcp_server.py                  → adds format_result(); deep_research(
```

Step 4 — A Timeout and a Partial-Result Fallback

Project: MCPResearch-Multi-Agent-Pipeline-Exposed-As-One-MCP-Tool — Step 4 of 4: A Timeout and a Partial-Result Fallback

What this step does: adds a hard timeout around the whole pipeline call, and makes `deep_research` return something useful — the best draft found so far, clearly marked as unconfirmed — instead of hanging the caller or returning nothing, if the pipeline runs too long or a step fails outright. **Why this step matters:** `MAX_REVISION_ROUNDS` (Step 1) stops the *loop* from running forever. It does not stop one *round* from taking a long time — a slow model response, a stuck network call — and it does nothing at all if `run_researcher` or `run_fact_checker` raises an exception. A tool with no timeout and no failure handling can hang a real client indefinitely, or crash with no useful information at all. **When you'll hit this for real:** any tool call — MCP or otherwise — that wraps something slow or unreliable needs both of these: a ceiling on how long you'll wait, and a real decision about what "the best answer I can give right now" looks like when that ceiling is hit. **Read first:** [11_multi_agent_systems Core Concepts](#) — "Error propagation between agents," and "Every pattern below: what / why / when / trade-off" (the Generator → Critic → Revision entry's "without a hard limit" trade-off, now applied to wall-clock time instead of just round count).

Stuck on this step? [Hint 1](#) · [Hint 2](#) · [Show me the solution](#)

What to do: 1. Add `DEEP_RESEARCH_TIMEOUT_SECONDS` to `config.py`. Write `exceptions.py` → `PipelineStepFailedError(Exception)`. In `pipeline.py`, wrap the Researcher/Fact-Checker calls in a `try / except`, and re-raise anything unexpected as `PipelineStepFailedError` with a

clear message naming which round failed — the same "turn a raw exception into a clean, informative error" idea Doc06 taught for one tool call, now applied to one pipeline round. 2. Give `run_pipeline()` an optional `progress_sink` argument — a callback invoked with each `RevisionRound` right after it's produced. This is what lets `mcp_server.py` see partial progress even if the whole call never finishes. 3. Make `deep_research` an `async def`. Inside it, run the pipeline with `asyncio.to_thread(run_pipeline, topic, config.max_revision_rounds, record_round)`, wrapped in `asyncio.wait_for(..., timeout=config.timeout_seconds)`. Catch `asyncio.TimeoutError` and `PipelineStepFailedError` separately, and in both cases return a clearly-marked partial result built from whatever `record_round` collected before the failure — never a bare error, never a hang. 4. **Know the real limitation before you test it:** `asyncio.wait_for()` stops *waiting*, it does not kill the background thread. A truly stuck `run_pipeline()` call keeps running after your timeout fires — it's just orphaned. This is a genuine, honest limit of this approach, not a bug to chase down; a real production system would run the pipeline in a separate process it can actually terminate, or design the loop to check a cancellation flag between rounds. Say this out loud once you've built it — it's a real trade-off, not something to paper over. 5. Test all three unhappy paths: a topic you make artificially slow (confirm the timeout fires and you get a marked-unconfirmed partial answer, not a hang), a forced exception inside `run_researcher` (confirm you get a marked-unconfirmed partial answer, not a crash or an empty string), and the happy path once more (confirm nothing about the normal result changed).

Your files after Step 4 (final):

```
project_12_mcpresearch_agentic_mcp_tool/
├── .env / .env.example           → adds DEEP_RESEARCH_TIMEOUT_SECONDS
├── config.py                     → now also loads DEEP_RESEARCH_TIMEOUT
├── logging_setup.py
├── exceptions.py                 → PipelineStepFailedError(Exception)
├── researcher.py                 (unchanged from Step 1)
├── fact_checker.py              (unchanged from Step 1)
├── pipeline.py                  → run_pipeline() now takes an
├── main.py                      (unchanged from Step 1)
└── mcp_server.py                → deep_research() is now a
```

Final Deliverable: MCPResearch-Multi-Agent-Pipeline-Exposed-As-One-MCP-Tool — a real MCP server exposing one Tool, `deep_research`, whose body runs a bounded Researcher/Fact-Checker revision loop, reports a per-round trace alongside the final answer, and falls back to a clearly-marked partial result instead of hanging or crashing when the pipeline runs too long or a step fails.

Checklist Before You Call This Done

- `[ ] researcher.py` and `fact_checker.py` work correctly on their own, tested through `main.py`, before any MCP code exists

- [] The revision loop actually improves a deliberately wrong first draft, and stops cleanly at `MAX_REVISION_ROUNDS` on a topic that never fully converges
- [] `deep_research` is a single MCP tool — the Inspector shows exactly one tool, not one per agent
- [] `deep_research` 's result includes a per-round revision trace, not just the bare final summary
- [] A slow or stuck pipeline run times out, and still returns a clearly-marked, useful partial answer instead of hanging the client
- [] A forced failure inside a pipeline step is caught and turned into a clear, marked-unconfirmed result, never a crash or an empty string
- [] No `print()` anywhere in `researcher.py` , `fact_checker.py` , `pipeline.py` , or `mcp_server.py` — only the logger
- [] You can explain, out loud, why this is one MCP tool wrapping two agents instead of two MCP tools a client drives itself

Status

Not started. Track your progress in [../PROGRESS.md](#).

Stuck? Ask for **Hint 1** or **Hint 2** about the exact part you're stuck on. Only ask for the full code if you say **"Show me the solution."**